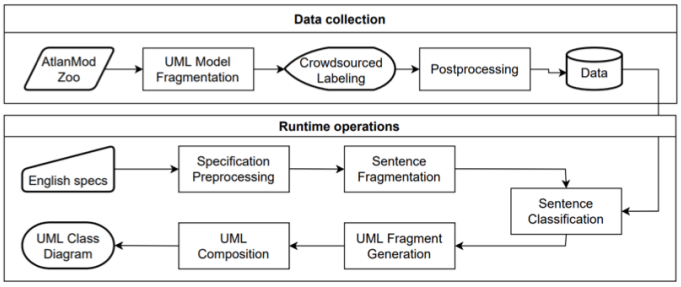
Plan & Design

PLAN & DESIGN

Natural language specification -> UML



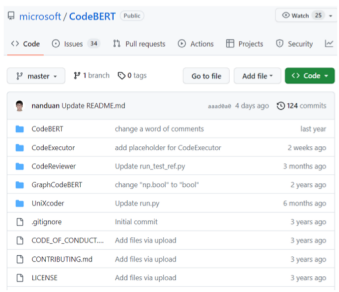
https://dl.acm.org/doi/abs/10.1145/3550356.3561592

Coding & Debugging

CODEBERT

- Training data: GitHub repos. Code paired with NLP documentation
- Model architecture: RoBERTa-base (multi-layer bi-directional transformer)
- Downstream tasks:
  - Code-to-text: generate documentation by code
  - Text-to-code: search code by text

https://arxiv.org/abs/2002.08155



CODEBERT Text -> Code

Query

create file and write something

Search Results (top2)

https://github.com/darknessomi/musicbox/blob/master/NEMbox/utils.py#L37-L40

```
def create_file(path, default="\n"):
    if not os.path.exists(path):
        with open(path, "w") as f:
            f.write(default)
```

https://github.com/datakortet/yamldirs/blob/master/yamldirs/filemaker.py#L114-L118

```
def make_file(self, filename, content):
    """Create a new file with name ``filename`` and content ``content``.
    """
    with open(filename, 'w') as fp:
        fp.write(content)
```

Given NL, search PL

CODEBERT Code -> Text

```
public void addWriteErrorResult(final BulkWriteError writeError,
        final IndexMap indexMap) {
    notNull("writeError", writeError);
    mergeWriteErrors(asList(writeError), indexMap);
}
```

Gold: Add a write error result

CodeBERT: Add a write error result .

PRE-TRAIN W/ CODEONLY : Merges the given write error .

Roberta: Add a write operation to the map .

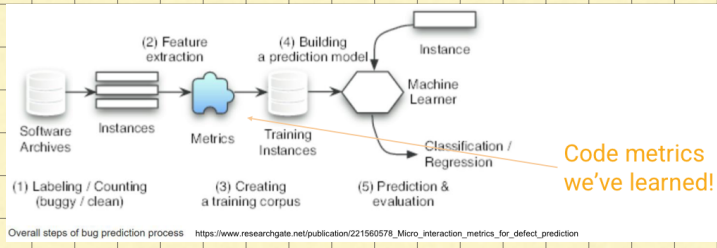
Transformer: Adds an error to the write map .

RNN: Add an error map .

Given PL, get NL

Figure 6: Java code documentation generation output example.

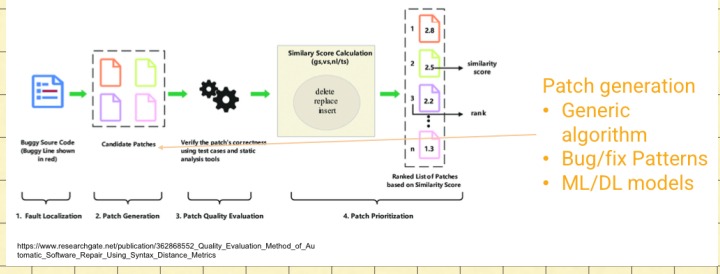
(Traditional) Bug Prediction



Code metrics we've learned!

Overall steps of bug prediction process https://www.researchgate.net/publication/221560578_Micro_interaction_metrics_for_defect_prediction

(Traditional) Automated Program Repair (APR)



Patch generation

- Generic algorithm
- Bug/fix Patterns
- ML/DL models

https://www.researchgate.net/publication/362868552_Quality_Evaluation_Method_of_Automatic_Software_Repair_Using_Syntax_Distance_Metrics

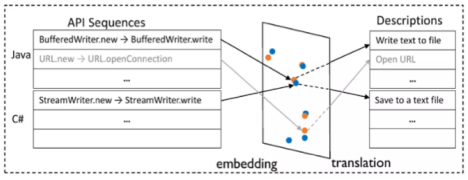
API Migration

– Motivation

- A software product is often required to support a variety of devices and environments.
- This requires **developing the software product in one language and manually porting it to other languages**.
- The procedure is tedious, time-consuming, and error-prone

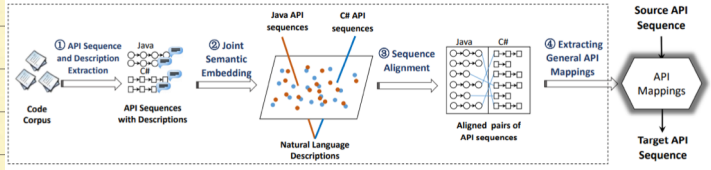
Reference: Gu Xiaodong, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. "DeepAM: Migrate APIs with multi-modal sequence to sequence learning." arXiv preprint arXiv:1704.07734 (2017).

– Approach



DEEPAM assigns to each API sequence a continuous vector in a high-dimensional semantic space in such a way that API sequences with similar vectors, or "embeddings", tend to have similar natural language descriptions.

Reference: Gu Xiaodong, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. "DeepAM: Migrate APIs with multi-modal sequence to sequence learning." arXiv preprint arXiv:1704.07734 (2017).



Reference: Gu Xiaodong, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. "DeepAM: Migrate APIs with multi-modal sequence to sequence learning." arXiv preprint arXiv:1704.07734 (2017).

– Results

parse datetime from string	
SimpleDateFormat.new→SimpleDateFormat.parse	DateTimeFormatInfo.new→DateTime.parseExact→DateTime.parse
open a url	
URL.new→URL.openConnection	WebRequest.create→Uri.new→HttpWebRequest.getRequestStream
get files in folder	
File.new→File.list→File.new→File.isDirectory	DirectoryInfo.new→DirectoryInfo.getDirectories
create a directory	
File.new→File.exists→File.createNewFile	FileInfo.new→Directory.exists→Directory.createDirectory

Reference: Gu Xiaodong, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. "DeepAM: Migrate APIs with multi-modal sequence to sequence learning." arXiv preprint arXiv:1704.07734 (2017).

Auto-update Library Dependencies

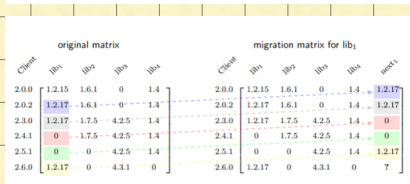
Table 1: Migration path of the *org.apache.hadoop:hadoop-auth* repository.

Client version	lib_1	lib_2	lib_3	lib_4	Timestamp
2.0.2-alpha	1.2.17	1.6.1	0	1.4	2012-10-02T00:44:04
2.3.0	1.2.17	1.7.5	4.2.5	1.4	2014-02-11T13:55:58
2.4.1	0	1.7.5	4.2.5	1.4	2014-06-21T06:08:34
2.5.1	0	0	4.2.5	0	2014-09-05T23:05:15
2.6.0	1.2.17	0	4.3.1	0	2014-11-13T22:35:37
2.7.2	1.2.17	1.7.10	4.2.5	1.4	2016-01-14T21:32:14
3.0.0-alpha3	1.2.17	1.7.10	4.5.2	1.4	2017-05-26T20:39:35
*	?	?	?	?	

- A software may depend on multiple 3rd-party libraries (TPL)
- Cannot simply upgrade to the next or latest version
- May involve library removal or downgrading
- It's complex to upgrade TPL versions without introducing compatibility issues

- A DL model can learn upgrades for TPLs that have been performed by similar clients.
- Such upgrades are considered safe, i.e., they do not trigger any conflict, since in the training clients, the libraries already coexist without causing any compatibility or dependency issues

Nguyen, Phuong T., et al. "DeepLib: Machine translation techniques to recommend upgrades for third-party libraries." Expert Systems with Applications 202 (2022): 117267.



- Data is collected from GitHub and Maven repository
- Migration matrices are built, normalized, and used as training data
- "next" is the label to be predicted
- LSTM model is used

Nguyen, Phuong T., et al. "DeepLib: Machine translation techniques to recommend upgrades for third-party libraries." Expert Systems with Applications 202 (2022): 117267.

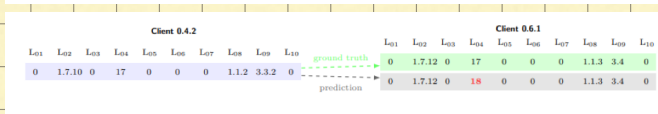


Figure 12: Recommendation for the *com.hubspot:SingularityService* repository.

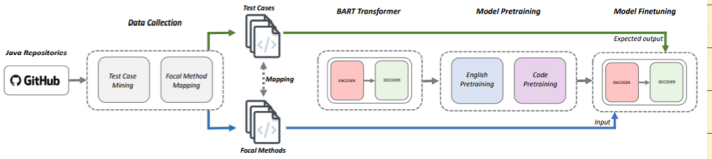
Nguyen, Phuong T., et al. "DeepLib: Machine translation techniques to recommend upgrades for third-party libraries." Expert Systems with Applications 202 (2022): 117267.

Auto-Generating Unit Tests

- Task: given a focal method (the method to be tested by the test case), generate a set of test cases
- Idea: learn from large test cases and corresponding focal methods

- Data: 91K Java projects from GitHub
- Identify and pair tests & focal methods using heuristics such as name matching

- BART transformer model
- English pretraining: 160GB English text from books, Wikipedia, and news articles
- Code pretraining: 25GB Java projects from GitHub
- Finetuning: the source is a focal method, and the target is the corresponding test case originally written by a software developer.



Tufano, Michele, et al. "Unit test case generation with transformers and focal context." arXiv preprint arXiv:2009.05617 (2020).